

Interface Segregation Principle in C# with Examples

In this article, I will discuss the **Interface Segregation Principle in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the [Liskov Substitution Principle in C#](#) with Examples. The letter **I** in the SOLID Design Principle stands for **Interface Segregation Principle**, also known as **ISP**. As part of this article, we will discuss the following pointers in detail.

1. [What is the Interface Segregation Principle in C#?](#)
2. [Example without using the Interface Segregation Principle in C#.](#)
3. [Example using the Interface Segregation Principle in C#.](#)
4. [How to Use Interface Segregation Principle in C#?](#)
5. [Advantages of Interface Segregation Principle in C#.](#)

What is the Interface Segregation Principle in C#?

The Interface Segregation Principle (ISP) is another SOLID principle of object-oriented design. This Principle states **Clients should not be forced to implement any methods they don't use. Rather than one fat interface, numerous little interfaces are preferred based on groups of methods, with each interface serving one submodule.**

Let us break down the above definition into two parts.

1. First, no class should be forced to implement any method(s) of an interface they don't use.
2. Secondly, instead of creating large or, you can say, fat interfaces, create multiple smaller interfaces so that the clients only think about the methods that they want to implement.

Example to Understand Interface Segregation Principle in C#.

Let us understand the Interface Segregation Principle in C# with an example. Please have a look at the following diagram. Here, we have one interface and two classes implementing that Interface.

```
public interface IPrinterTasks
{
    void Print(string PrintContent);
    void Scan(string ScanContent);
    void Fax(string FaxContent);
    void PrintDuplex(string PrintDuplexContent);
}
```

Forcing a class to provide the body of an interface method means violating the Interface Segregation Principle

```
class HPLaserJetPrinter : IPrinterTasks
{
    public void Print(string PrintContent)
    {
        Console.WriteLine("Print Done");
    }

    public void Scan(string ScanContent)
    {
        Console.WriteLine("Scan content");
    }

    public void Fax(string FaxContent)
    {
        Console.WriteLine("Fax content");
    }

    public void PrintDuplex(string PrintDuplexContent)
    {
        Console.WriteLine("Print Duplex content");
    }
}
```

```
class LiquidInkjetPrinter : IPrinterTasks
{
    public void Print(string PrintContent)
    {
        Console.WriteLine("Print Done");
    }

    public void Scan(string ScanContent)
    {
        Console.WriteLine("Scan content");
    }

    public void Fax(string FaxContent)
    {
        throw new NotImplementedException();
    }

    public void PrintDuplex(string PrintDuplexContent)
    {
        throw new NotImplementedException();
    }
}
```

As you can see in the above diagram, we have an interface, i.e., **IPrinterTasks** declared with four methods. Now, if any class wants to implement this interface, then that class should and must have to provide the implementation to all four methods of the **IPrinterTasks** interface. As you can see in the above diagram, we have two classes, **HPLaserJetPrinter** and **LiquidInkjetPrinter**, who want the printer service.

But the requirement is the **HPLaserJetPrinter** wants all the services provided by the **IPrinterTasks** while the **LiquidInkjetPrinter** wants only the Print and Scan service of the printer. As we have declared all the methods within the **IPrinterTasks** interface, the **LiquidInkjetPrinter** class must provide implementation to Scan and Print methods along with the Fax and PrintDulex methods, which are not required by the class. This violates the Interface Segregation Principle in C#, forcing the class to provide the implementation they don't require.

Example Without using the Interface Segregation Principle in C#:

Let us first see the example without following the Interface Segregation Principle, and then we will see the problem. and finally, we will rewrite the same example by following Interface Segregation Principle using C# Language.

IPrinterTasks.cs

Create a class file named **IPrinterTasks.cs**, then copy and paste the following code. Here, **IPrinterTasks** is an interface that contains the declaration of four methods. By default, interface methods are public and

abstract, so the child class of this interface needs to implement all these methods.

```
namespace SOLID_PRINCIPLES.ISP
{
    public interface IPrinterTasks
    {
        void Print(string PrintContent);
        void Scan(string ScanContent);
        void Fax(string FaxContent);
        void PrintDuplex(string PrintDuplexContent);
    }
}
```

HPLaserJetPrinter.cs

Next, create a class file with the name **HPLaserJetPrinter.cs** and copy and paste the following code. Here, the HPLaserJetPrinter implements the IPrinterTasks interface and provides implementations for all four methods. This class requires all printer services, so there is no issue in implementing all interface methods.

```
using System;
namespace SOLID_PRINCIPLES.ISP
{
    public class HPLaserJetPrinter : IPrinterTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine("Print Done");
        }
        public void Scan(string ScanContent)
        {
            Console.WriteLine("Scan content");
        }
        public void Fax(string FaxContent)
        {
            Console.WriteLine("Fax content");
        }
        public void PrintDuplex(string PrintDuplexContent)
        {
            Console.WriteLine("Print Duplex content");
        }
    }
}
```

LiquidInkjetPrinter.cs

Next, create a class file named **LiquidInkjetPrinter.cs** and copy and paste the following code. Here, the LiquidInkjetPrinter implements the IPrinterTasks interface and provides implementations for all four methods. But, this class only requires the Print and Scan services. This class does not require Fax and PrintDuplex services but still implements these two methods. This violates the Interface Segregation Principle as we force the class to implement two methods they don't require.

```

using System;
namespace SOLID_PRINCIPLES.ISP
{
    class LiquidInkjetPrinter : IPrinterTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine("Print Done");
        }
        public void Scan(string ScanContent)
        {
            Console.WriteLine("Scan content");
        }
        public void Fax(string FaxContent)
        {
            throw new NotImplementedException();
        }
        public void PrintDuplex(string PrintDuplexContent)
        {
            throw new NotImplementedException();
        }
    }
}

```

Why are we Facing this problem?

We face the above problem because we declare all the methods in a single class. As per Inheritance Rules, the child class will implement all interface methods. Because of this, the LiquidInkjetPrinter class provides implementation to all IPrinterTasks Interface methods. We can overcome this problem by following the Interface Segregation Principle. Let us proceed and try to understand how we can rewrite the same example by following the Interface Segregation Principle using C# Language.

Example using Interface Segregation Principle in C#:

Please have a look at the following image. As you can see in the image below, we have split that big interface into three small interfaces. Each interface now has some specific purpose.

```

public interface IPrinterTasks
{
    void Print(string PrintContent);
    void Scan(string ScanContent);
}
interface IFaxTasks
{
    void Fax(string content);
}
interface IPrintDuplexTasks
{
    void PrintDuplex(string content);
}

```

Splitting a big interface into smaller ones

Now, if any class wants all the printer services, then that class needs to implement all three interfaces. In our example, HPLaserJetPrinter wants all the printer services. So, the HPLaserJetPrinter class needs to implement all three interfaces and provide implementations of all interface methods, as shown in the below image.

```

public class HPLaserJetPrinter : IPrinterTasks, IFaxTasks,
                                IPrintDuplexTasks
{
    public void Print(string PrintContent) ...
    public void Scan(string ScanContent) ...
    public void Fax(string FaxContent) ...
    public void PrintDuplex(string PrintDuplexContent) ...
}

```

Class implementing all the three interfaces means this class wants all the four services

Now, if any class wants the Scan and Print services, then that class needs to implement only the IPrinterTasks interfaces. In our example, LiquidInkjetPrinter wants only the Scan and Print services. So, the LiquidInkjetPrinter class must implement only the IPrinterTasks interfaces and provide implementations for Print and Scan methods, as shown in the image below.

```
class LiquidInkjetPrinter : IPrinterTasks
{
    public void Print(string PrintContent) ...
    public void Scan(string ScanContent) ...
}
```

This class wants only Print and Scan service so implementing only the IPrinterTasks interface

The Complete Example Code is Given Below:

First, modify the IPrinterTasks.s class file as follows. Here, we are splitting the big interface into three interfaces. Each interface has a specific purpose. Based on the requirement, the child class will implement one, two, or all the interfaces.

```
namespace SOLID_PRINCIPLES.ISP
{
    public interface IPrinterTasks
    {
        void Print(string PrintContent);
        void Scan(string ScanContent);
    }
    interface IFaxTasks
    {
        void Fax(string content);
    }
    interface IPrintDuplexTasks
    {
        void PrintDuplex(string content);
    }
}
```

Next, modify the HPLaserJetPrinter.cs class file as follows. The HPLaserJetPrinter printer class requires all the printer services, hence implementing all three interfaces and providing implementation to all four methods.

```
using System;
namespace SOLID_PRINCIPLES.ISP
{
    public class HPLaserJetPrinter : IPrinterTasks, IFaxTasks, IPrintDuplexTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine("Print Done");
        }
        public void Scan(string ScanContent)
        {

```

```

        Console.WriteLine("Scan content");
    }
    public void Fax(string FaxContent)
    {
        Console.WriteLine("Fax content");
    }
    public void PrintDuplex(string PrintDuplexContent)
    {
        Console.WriteLine("Print Duplex content");
    }
}
}

```

Next, modify the LiquidInkjetPrinter.cs class file as follows. The LiquidInkjetPrinter printer class requires only the Print and Scan Printer services and hence implements only the IPrinterTasks interface and provides implementation to Print and Scan methods.

```

using System;
namespace SOLID_PRINCIPLES.ISP
{
    class LiquidInkjetPrinter : IPrinterTasks
    {
        public void Print(string PrintContent)
        {
            Console.WriteLine("Print Done");
        }
        public void Scan(string ScanContent)
        {
            Console.WriteLine("Scan content");
        }
    }
}

```

Now, you can see the LiquidInkjetPrinter class is not implementing the Fax and PrintDuplex methods as these services are not required by the LiquidInkjetPrinter class. That means now our application design follows the Interface Segregation Principle. Now, you can test the functionality of the two classes by modifying the Program class code as follows.

```

using System;
namespace SOLID_PRINCIPLES.ISP
{
    public class Program
    {
        static void Main(string[] args)
        {
            //Using HPLaserJetPrinter we can access all Printer Services
            HPLaserJetPrinter hPLaserJetPrinter = new HPLaserJetPrinter();
            hPLaserJetPrinter.Print("Printing");
            hPLaserJetPrinter.Scan("Scanning");
            hPLaserJetPrinter.Fax("Faxing");
            hPLaserJetPrinter.PrintDuplex("PrintDuplex");
        }
    }
}

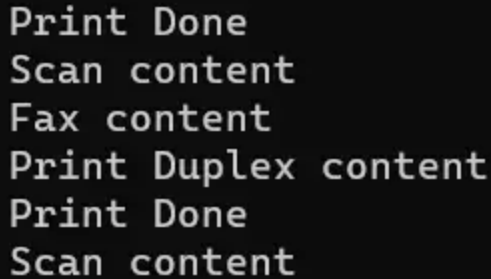
```

```

        //Using LiquidInkjetPrinter we can only Access Print and Scan Pri
Services
        LiquidInkjetPrinter liquidInkjetPrinter = new LiquidInkjetPrinter(
        liquidInkjetPrinter.Print("Printing");
        liquidInkjetPrinter.Scan("Scanning");
        Console.ReadKey());
    }
}

```

When you run the above code, you will get the following output.



```

Print Done
Scan content
Fax content
Print Duplex content
Print Done
Scan content

```

Note: A solution to having a better code always comes down to a word small: small method, small class, small interface. So, if you split your code into smaller chunks, it will be easier to change and maintain it.

Advantages of Interface Segregation Principle in C#:

The following are some of the key advantages of applying the Interface Segregation Principle in C#:

- **Reduced Side Effects:** By segregating interfaces, changes to one part of a system are less likely to affect components that implement these interfaces. Clients will only know about the methods that are relevant to them, so changes in unrelated methods won't impact them.
- **Increased Clarity and Focus:** Smaller, well-defined interfaces are easier to understand and implement. Each interface focuses on a specific aspect, which helps developers quickly grasp what's required without sifting through irrelevant functionality.
- **Easier Maintenance and Extension:** With smaller, segregated interfaces, maintaining and extending software becomes more straightforward. Developers can add new functionalities without modifying existing interfaces simply by creating new interfaces and having classes implement multiple interfaces as needed.
- **Enhanced Reusability:** Segregated interfaces enhance reusability. Since each interface is smaller and more specific, it can be more easily reused across different parts of a system or even in different projects without carrying unnecessary dependencies.
- **Promotes Decoupling:** ISP helps to reduce the coupling between classes. It allows implementing classes to only depend on the interfaces that they actually use, which minimizes the dependencies between components.

In the next article, I will discuss [Multiple Real-Time Examples of the Interface Segregation Principle \(ISP\) in C#](#). In this article, I explain the Interface Segregation Principle in C# with Examples. I hope you enjoy this Interface Segregation Principle in C# with Examples article.